Thus, any C program we'll ever need to build can be constructed from only seven control statements combined in only two ways. This is the essence of simplicity.

Self Check

1 *(Multiple Choice)* Which of the following statements is *false*?
   a) Normally, statements in a program execute one after the other in the order in which they're written. This is called transfer of control.
   b) Programs can be be written without any `goto` statements.
   c) All programs can be written in terms of only three control structures—the sequence structure, the selection structure and the iteration structure.
   d) The sequence structure is simple—unless directed otherwise, the computer executes C statements one after the other in the order in which they're written.

Answer: a) is *false*. It's actually called sequential execution.

2 *(Multiple Choice)* Which of the following statements is *false*?
   a) C has only seven control statements: sequence, three types of selection and three types of iteration.
   b) Single-entry/single-exit control statements make it easy to build clear programs.
   c) Connecting the exit point of one control statement to the entry point of the next is called control-statement stacking.
   d) The only other way to connect control statements is via roosting. Thus, any C program we'll ever need to build can be constructed from only seven different types of control statements combined in only two ways.

Answer: d) is *false*. The only other way to connect control statements is via nesting.

3.5 The `if` Selection Statement

Selection statements choose among alternative courses of action. For example, suppose the passing grade on an exam is 60. The following pseudocode statement determines whether the condition "student's grade is greater than or equal to 60" is *true* or *false*:

```
If student's grade is greater than or equal to 60
    Print "Passed"
```

If *true*, then "Passed" is printed, and the next pseudocode statement in order is "performed." Remember that pseudocode isn't a real programming language. If *false*, the printing is ignored, and the next pseudocode statement in order is performed.

The preceding pseudocode is written in C as

```
if (grade >= 60) {
   puts("Passed");
} // end if
```

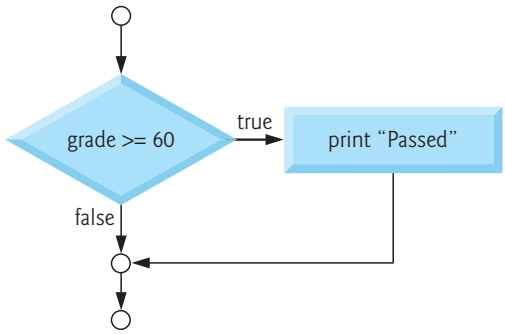
Of course, you'll also need to declare the `int` variable `grade`, but the C `if` statement code corresponds closely to the pseudocode. This is one of the properties of pseudocode that makes it such a useful program-development tool.

Indentation in the `if` Statement

The indentation in the `if` statement's second line is optional but highly recommended. It emphasizes the inherent structure of structured programs. The compiler ignores **white-space characters** such as blanks, tabs and newlines used for indentation and vertical spacing.

`if` Statement Flowchart

The following flowchart segment illustrates the single-selection `if` statement:



It contains perhaps the most important flowchart symbol—the **diamond (or decision) symbol,** which indicates a decision is to be made. The decision symbol's expression typically is a condition that can be *true* or *false*. The two flowlines emerging from it indicate the paths to take when the expression is *true* or *false*. Decisions can be based on any expression's value—zero is *false*, and nonzero is *true*.

The `if` statement is a single-entry/single-exit statement. We'll soon learn that the flowchart segment for the remaining control structures also can contain rectangle symbols to indicate the actions to be performed and diamond symbols to indicate decisions to be made. This is the action/decision model of programming we've been emphasizing.

Self Check

1 *(Multiple Choice)* Which of the following statements is *false*?

a) The pseudocode statement

If student's grade is greater than or equal to 60
  Print "Passed"

can be written in C as

```
if (grade >= 60) {
   puts("Passed");
} // end if
```

b) The two flowlines emerging from the decision flowchart symbol indicate the directions to take when the expression in the symbol is *true* or *false*.

c) Decisions can be based on any expression—if the expression evaluates to nonzero, it's treated as *false*, and if it evaluates to zero, it's treated as *true*.

d) The `if` statement is a single-entry/single-exit statement.

Answer: c) is *false*. Actually, decisions can be based on any expression—if the expression evaluates to zero, it's treated as *false*, and if it evaluates to nonzero, it's treated as *true*.

3.6 The `if...else` Selection Statement

The `if...else` selection statement specifies different actions to perform when the condition is *true* or *false*. For example, the pseudocode statement

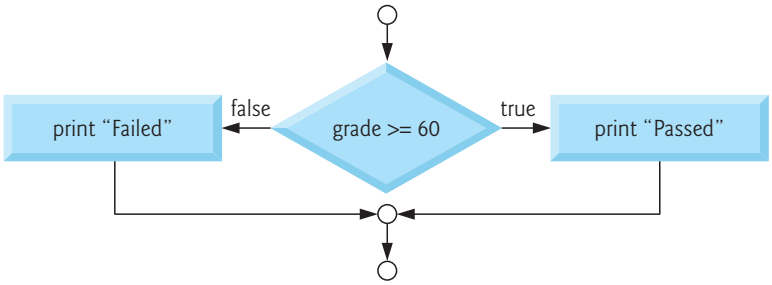
```
If student's grade is greater than or equal to 60
    Print "Passed"
else
    Print "Failed"
```

prints "Passed" if the student's grade is greater than or equal to 60; otherwise, it prints "Failed." In either case, after printing, the next pseudocode statement in sequence "executes." The *else*'s body also is indented. If there are several levels of indentation in a program, each should be indented the same additional amount of space. The preceding pseudocode may be written in C as

```
if (grade >= 60) {
   puts("Passed");
} // end if
else {
   puts("Failed");
} // end else
```

`if...else` Statement Flowchart

The following flowchart illustrates the `if...else` statement's flow of control:



Conditional Expressions

The **conditional operator** (`?:`) is closely related to the `if...else` statement. This operator is C's only **ternary operator**—that is, it takes three operands. A conditional operator and its three operands form a **conditional expression**. The first operand is a condition. The second is the conditional expression's value if the condition is *true*. The third is the conditional expression's value if the condition is *false*. For example, the conditional-expression argument to the following `puts` statement evaluates to the string `"Passed"` if the condition `grade >= 60` is true; otherwise, it evaluates to the string `"Failed"`:

```
puts((grade >= 60) ? "Passed" : "Failed");
```

Conditional operators can be used in places where `if...else` statements cannot, including expressions and arguments to functions (such as `printf`). Use expressions of the same type for the second and third operands of the conditional operator (`?:`) to avoid subtle errors.

ERR

Nested if...else Statements

Nested if...else statements test for multiple cases by placing `if...else` statements inside `if...else` statements. For example, the following pseudocode statement prints: `A` for grades greater than or equal to `90`, `B` for grades greater than or equal to `80` (but less than `90`), `C` for grades greater than or equal to `70` (but less than `80`), `D` for grades greater than or equal to `60` (but less than `70`), and `F` for all other grades.

```
If student's grade is greater than or equal to 90
    Print "A"
else
    If student's grade is greater than or equal to 80
        Print "B"
    else
        If student's grade is greater than or equal to 70
            Print "C"
        else
            If student's grade is greater than or equal to 60
                Print "D"
            else
                Print "F"
```

This pseudocode may be written in C as

```
if (grade >= 90) {
   puts("A");
} // end if
else {
   if (grade >= 80) {
      puts("B");
   } // end if
   else {
      if (grade >= 70) {
         puts("C");
      } // end if
      else {
         if (grade >= 60) {
            puts("D");
         } // end if
         else {
            puts("F");
         } // end else
      } // end else
   } // end else
} // end else
```

If the variable `grade` is greater than or equal to 90, all four conditions are true, but only the `puts` statement after the first test executes. Then, the `else` part of the "outer" `if...else` statement is skipped, bypassing the rest of the nested `if...else` statement.

Most programmers write the preceding `if` statement as

```
if (grade >= 90) {
   puts("A");
} // end if
else if (grade >= 80) {
   puts("B");
} // end else if
else if (grade >= 70) {
   puts("C");
} // end else if
else if (grade >= 60) {
   puts("D");
} // end else if
else {
   puts("F");
} // end else
```

Both forms are equivalent. The latter form avoids the deep indentation to the right, which decreases program readability and sometimes causes lines to wrap.

Blocks and Compound Statements

To include several statements in an `if`'s body, you must enclose the statements in braces (`{` and `}`). A set of statements contained within a pair of braces is called a **compound statement** or a **block**. A compound statement can be placed anywhere in a program that a single statement can be placed.

The following `if...else` statement's `else` part includes a compound statement containing two statements to execute if the condition is *false*:

```
if (grade >= 60) {
   puts("Passed.");
} // end if
else {
   puts("Failed.");
   puts("You must take this course again.");
} // end else
```

If `grade` is less than `60`, both `puts` statements in the `else` execute and the code prints:

```
Failed.
You must take this course again.
```

The braces surrounding the two statements in the `else` clause are important. Without them, the statement

```
puts("You must take this course again.");
```

would be outside the `else`'s body (and outside the `if...else` statement) and would execute regardless of whether the grade was less than 60, so even a passing student would have to take the course again. To avoid problems like this, always include your control statements' bodies in braces (`{` and `}`), even if those bodies contain only a single statement. This solves the "dangling-else" problem, which we discuss in this chapter's exercises.

Kinds of Errors

A syntax error (such as misspelling "else") is caught by the compiler. A logic error has its effect at execution time. A fatal logic error causes a program to fail and terminate prematurely. A nonfatal logic error allows a program to continue executing but to produce incorrect results. ERR

Empty Statement

Anywhere a single or compound statement can be placed, it's possible to place an empty statement, represented by a semicolon (;). Placing a semicolon after an if's condition, as in ERR

```
if (grade >= 60);
```

leads to a logic error in single-selection if statements and a syntax error in double-selection and nested if...else statements.

Type both braces of compound statements before typing the individual statements within the braces. This helps avoid omission of one or both of the braces, preventing syntax errors (such as an if statement whose if part has multiple statements, which requires a pair of braces) and logic errors. Many integrated development environments and code editors insert the closing brace for you as soon as you type the opening one.

Self Check

1 *(True/False)* The following code includes a compound statement in an if...else statement's else part:

```
if (grade >= 60) {
   puts("Passed.");
} // end if
else
   puts("Failed.");
   puts("You must take this course again.");
```

Answer: *False.* The curly braces around the two puts statements in the else part of this if...else statement are missing. The correct code with the compound statement is

```
if (grade >= 60) {
   puts("Passed.");
} // end if
else {
   puts("Failed.");
   puts("You must take this course again.");
} // end else
```

2 *(True/False)* The conditional expression argument to the following puts statement

```
puts((grade >= 60) : "Passed" ? "Failed");
```

evaluates to the string "Passed" if the condition grade >= 60 is *true*; otherwise, it evaluates to the string "Failed".